

Bausteine Computergestützter Datenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Franca Hollmann
Maik Poetzsch	Sebastian Seipel	

2026-09-17

Table of contents

Python Tool Module	4
Prerequisites	4
Learning Objectives	4
1 Introduction	6
2 Welcome to Python!	7
2.1 Learning goals of this chapter	7
2.2 Your first program	7
2.3 Variables – Names for values	7
3 Understanding Data Types	9
3.1 Learning Objectives of This Chapter	9
3.2 Introduction	9
3.3 The Most Important Data Types	9
3.4 Difference Between <code>int</code> and <code>float</code>	10
3.5 What Are Booleans (<code>bool</code>)?	10
3.6 Doing Math with Numbers	11
3.7 Working with Text	12
3.8 Type Conversion (Typecasting)	12
4 Decisions and Loops	14
4.1 Learning Objectives of This Chapter	14
4.2 Conditions with <code>if</code> , <code>elif</code> , <code>else</code>	14
4.3 Comparison Operators	15
4.4 Repetitions with <code>while</code>	15
4.5 Loops with <code>for</code> and <code>range()</code>	16
4.6 What exactly does <code>range()</code> do?	16
4.6.1 Variants:	16
5 Storing Multiple Values	19
5.1 What is a List?	19
5.2 Slicing Lists	19
5.2.1 Syntax: <code>list[start:stop]</code>	20
5.3 Iterating Over Lists	20
5.4 Extension: Conditional Output	21

5.5	Calculating the Average	21
5.6	Expanding Lists: <code>.append()</code>	21
5.7	Nested Loops	22
5.8	Sorting Lists	22
6	Reusable Code with Functions	24
6.1	Learning Goals of This Chapter	24
6.2	Defining a Function	24
6.3	Passing Parameters	25
6.4	Return Values with <code>return</code>	25
6.5	Example: Conversions	26
6.5.1	Euro to US Dollar	26
6.6	Parameters with Default Values	27
7	Working with Files	28
7.1	Learning Goals of This Chapter	28
7.2	Reading a File	28
7.3	Reading Line by Line	29
7.4	Writing to a File	29
7.5	Writing Line by Line Using a Loop	30
7.6	Reading All Lines at Once with <code>readlines()</code>	31
7.7	Closing Files Manually with <code>close()</code>	31

Python Tool Module



Bausteine Computergestützter Datenanalyse by Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch and Sebastian Seipel. Python Tool Module by Marc Fehr and Maik Poetzsch is licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). The work is available on [GitHub](https://github.com/bausteine-der-datenanalyse/w-python-minimal-en). Excluded from the license are all third-party logos and otherwise marked content. 2026

Suggested citation

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, and Sebastian Seipel. 2025. “Bausteine Computergestützter Datenanalyse. Python Tool Module”. <https://github.com/bausteine-der-datenanalyse/w-python-minimal-en>.

BibTeX template

```
@misc{BCD-w-python-2025,  
  title={Bausteine Computergestützter Datenanalyse. Python Tool Module},  
  author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Hollmann,  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/w-python-minimal-en}}
```

Prerequisites

No requirements

Learning Objectives

In this module, the fundamentals of programming with Python are taught. In this module, you will learn ...

- the basics of programming
- output in Python, basic data types, flow control

- how to read and use documentation
- how to import modules and packages

1 Introduction



Figure 1.1: Logo of the Python Programming Language

Python logo by the Python Software Foundation is licensed under the [GPLv3](https://www.gnu.org/licenses/gpl-3.0.html). The wordmark is a registered trademark: <https://www.python.org/psf/trademarks/>. The original can be found on [Wikimedia](https://commons.wikimedia.org/wiki/File:Python_logo.svg). 2008

2 Welcome to Python!

Python is a modern programming language that is especially well-suited for beginners. It is easy to understand and used in many fields – from data analysis to web development.

In this course, you will learn Python step by step through practical examples.

2.1 Learning goals of this chapter

By the end of this chapter, you will be able to: - write simple Python programs, - display text on the screen, - define and use basic variables.

2.2 Your first program

The first steps in any new programming language are always the same. We start by printing the words ‘Hello World’. For this, we use the `print()` command:

```
print("Hello World!")
```

Hello World!

What happens here? - `print()` is a **function** that displays something on the screen. - The text "Hello World!" is shown. - Texts (also called “strings”) are always enclosed in quotation marks.

2.3 Variables – Names for values

Variables are like labeled drawers: they store information under a name.

```
name = "Ms. Miller"  
age = 32
```

You can use these variables to create dynamic output:

```
print(name + " is " + str(age) + " years old.")
```

Ms. Miller is 32 years old.

Note that here you are trying to output both a number and text. Therefore, we use the `str()` function to convert the number into text.

Exercise: Greeting with age

Write a program that greets you with your name:

Hello Ms. Miller!

Tip: In Python, you can concatenate text using `+`. Remember that strings must be in quotation marks.

Solution

```
my_name = "Your name here"
print("Hello " + my_name + "!")
```

Hello Your name here!

Extend your program so that it greets you and also states your age:

Hello Ms. Miller!

You are 32 years old.

Tip: Use multiple `print()` statements or concatenate the text.

Solution

```
name = "Ms. Miller"
age = 32

print("Hello " + name + "!")
print("You are " + str(age) + " years old.")
```

Hello Ms. Miller!

You are 32 years old.

3 Understanding Data Types

3.1 Learning Objectives of This Chapter

By the end of this chapter, you will be able to:

- distinguish the most important data types,
- work with numbers and text,
- perform simple calculations and print outputs.

3.2 Introduction

Python has various **data types**. These describe **what kind of data** you store in variables. This is important because many operations—like `+`—can behave differently depending on the data type:

- `+` with numbers means **addition**,
- `+` with text means **concatenation** (joining together).

So before we move on to more complex programs, we should understand which data types exist and how to use them.

3.3 The Most Important Data Types

Here are the basic data types in Python:

Type	Example	Meaning
<code>int</code>	10	Integer (whole number)
<code>float</code>	3.14	Decimal number (floating point)
<code>str</code>	"Hello"	Text (string)
<code>bool</code>	True, False	Boolean (yes/no)

You can check the type of a variable using the `type()` function:

```
value = 42
print(type(value)) # Output: <class 'int'>
```

```
<class 'int'>
```

3.4 Difference Between int and float

In Python, we distinguish between **integers** (int) and **floating point numbers** (float):

- int stands for “integer” – whole numbers like 1, 0, -10
- float stands for “floating point number” – numbers with decimal places like 3.14, 0.5, -2.0

```
a = 10      # int
b = 2.5     # float

print("a:", a, "| Type:", type(a))
print("b:", b, "| Type:", type(b))
```

```
a: 10 | Type: <class 'int'>
b: 2.5 | Type: <class 'float'>
```

Important

This distinction is important: some operations behave slightly differently depending on the data type.

3.5 What Are Booleans (bool)?

A **Boolean** is a truth value: it can only have two possible states:

- True
- False

These values show up in situations like:

- Is the temperature above 30 °C?
- Does the file have a certain name?
- Is the list empty?

```
is_sunny = True
has_umbrella = False

print("Sunny:", is_sunny)
print("Has umbrella?", has_umbrella)
print("Type of 'is_sunny':", type(is_sunny))
```

```
Sunny: True
Has umbrella? False
Type of 'is_sunny': <class 'bool'>
```

Booleans are especially useful in **conditions** and **comparisons**, which you will learn more about in Chapter 3.

3.6 Doing Math with Numbers

Python can be used like a calculator:

Operator	Description
+, -	Addition / Subtraction
*, /	Multiplication / Division
//, %	Integer Division / Remainder
**	Exponentiation

```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Exponentiation:", a**b)
print("Division:", a / b)
print("Integer Division:", a // b)
print("Remainder:", a % b)
```

```
Addition: 13
Subtraction: 7
Multiplication: 30
```

Exponentiation: 1000
Division: 3.3333333333333335
Integer Division: 3
Remainder: 1

Note

// means integer division: the result is rounded down. Alternatively, % performs integer division and returns the remainder.

3.7 Working with Text

Text (strings) can be combined:

```
first_name = "Anna"  
last_name = "Example"  
print("Welcome, " + first_name + " " + last_name + "!")
```

Welcome, Anna Example!

If you want to combine text and numbers, you must convert the number to a string:

```
points = 95  
print("You scored " + str(points) + " points.")
```

You scored 95 points.

3.8 Type Conversion (Typecasting)

Sometimes you need to convert a value from one data type to another— e.g., a number to a string so it can be printed.

This is called **typecasting**. Here are the most common functions:

Function	Description	Example
<code>str()</code>	Number → Text	<code>str(42) → "42"</code>
<code>int()</code>	Text/number → Integer	<code>int("10") → 10</code>

Function	Description	Example
<code>float()</code>	Text/number → Float	<code>float("3.14")</code> → 3.14

```
# Example: display a number as text
points = 100
print("You scored " + str(points) + " points.")

# Example: convert string to number and do a calculation
input_value = "3.5"
value = float(input_value) * 2
print("Double the value:", value)
```

You scored 100 points.
Double the value: 7.0

When converting, make sure the content is appropriate – `int("abc")` will cause an error.

💡 Exercise: Age in Days

Calculate how old a person is in days.

```
age_years = 32
days = age_years * 365
print("You are approximately " + str(days) + " days old.")
```

You are approximately 11680 days old.

Tip: Remember to convert the number to a string when printing it.

Solution

```
age = 32
days = age * 365
print("You are approximately " + str(days) + " days old.")
```

You are approximately 11680 days old.

4 Decisions and Loops

Programs often need to make decisions – for example, based on user input or a specific value. Similarly, certain actions need to be repeated multiple times.

Python provides two key elements for this:

- **Control structures:** `if`, `elif`, `else`
- **Loops:** `while` and `for`

4.1 Learning Objectives of This Chapter

By the end of this chapter, you will be able to:

- Formulate conditions using `if`, `elif`, and `else`,
- Use comparison operators (`==`, `<`, `!=`, ...),
- Implement loops with `while` and `for`.

4.2 Conditions with `if`, `elif`, `else`

```
age = 17

if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

You are a minor.

Handling multiple cases:

```

grade = 2.3

if grade <= 1.5:
    print("Very good")
elif grade <= 2.5:
    print("Good")
elif grade <= 3.5:
    print("Satisfactory")
else:
    print("Sufficient or worse")

```

Good

4.3 Comparison Operators

Expression	Meaning
<code>a == b</code>	equal
<code>a != b</code>	not equal
<code>a < b</code>	less than
<code>a > b</code>	greater than
<code>a <= b</code>	less than or equal
<code>a >= b</code>	greater than or equal

4.4 Repetitions with while

```

counter = 0

while counter < 5:
    print("Counter is:", counter)
    counter += 1

```

```

Counter is: 0
Counter is: 1
Counter is: 2
Counter is: 3
Counter is: 4

```

Important

Make sure to include a stopping condition – otherwise, the loop will run endlessly!

4.5 Loops with for and range()

If you want to repeat something **a specific number of times**, use **for** with **range()**:

```
for i in range(5):  
    print("Iteration:", i)
```

```
Iteration: 0  
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4
```

Set a custom start and end value:

```
for i in range(1, 6):  
    print(i)
```

```
1  
2  
3  
4  
5
```

4.6 What exactly does range() do?

The **range()** function creates a sequence of numbers that can be iterated over with a **for** loop.

4.6.1 Variants:

```
range(5)
```

yields: 0, 1, 2, 3, 4 (starts at 0, ends **before** 5)

```
range(2, 6)
```

yields: 2, 3, 4, 5 (starts at 2, ends **before** 6)

```
range(1, 10, 2)
```

yields: 1, 3, 5, 7, 9 (step size = 2)

`range()` does not create a real list, but a so-called “range object” that behaves like a list.



Task: Count from 1 to 10

Use a `for` loop to print the numbers from 1 to 10.

Solution

```
for i in range(1, 11):  
    print(i)
```

1
2
3
4
5
6
7
8
9
10



Task: Print even numbers

Print all even numbers from 0 to 20. Tip: A number is even if `number % 2 == 0`.

Solution

```
for number in range(0, 21):  
    if number % 2 == 0:  
        print(number)
```

0
2
4
6
8
10
12
14
16
18
20

5 Storing Multiple Values

Until now, you've stored individual values in variables. But what if you want to store a whole list of numbers, names, or values at once?

For that, Python has **lists**. In this chapter, you'll also learn how to iterate over lists using **for** loops.

5.1 What is a List?

A list is an ordered collection of values of the same data type.

```
names = ["Ali", "Bente", "Carlos"]
grades = [1.7, 2.3, 1.3, 2.0]
```

You access elements using square brackets:

```
print(names[0])    # first element
print(grades[-1]) # last element
```

```
Ali
2.0
```

5.2 Slicing Lists

With **slicing**, you can extract specific parts of a list. You specify **where the slice starts and where it ends** (the end value is **not** included):

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4]) # Output: [20, 30, 40]
```

```
[20, 30, 40]
```

5.2.1 Syntax: list[start:stop]

- **start:** index where slicing starts (inclusive)
- **stop:** index where it ends (exclusive)
- You can omit the start index: `[:3]` → first to third element
- Or omit the stop index: `[3:]` → from fourth element to end
- Full copy: `[:]`

```
print(numbers[:3])    # [10, 20, 30]
print(numbers[3:])    # [40, 50, 60]
print(numbers[:])     # full copy
```

```
[10, 20, 30]
[40, 50, 60]
[10, 20, 30, 40, 50, 60]
```

Note

You can also use negative indices (`-1` is the last element):

```
print(numbers[-3:])  # [40, 50, 60]
```

```
[40, 50, 60]
```

5.3 Iterating Over Lists

You can use a `for` loop to iterate over each element in a list:

```
names = ["Ali", "Bente", "Carlos"]

for name in names:
    print("Hello", name + "!")
```

```
Hello Ali!
Hello Bente!
Hello Carlos!
```

5.4 Extension: Conditional Output

You can use `if` statements inside the loop to filter:

```
temperatures = [14.2, 17.5, 19.0, 21.3, 18.4]

for t in temperatures:
    if t > 18:
        print(t, "is a warm day")
```

```
19.0 is a warm day
21.3 is a warm day
18.4 is a warm day
```

5.5 Calculating the Average

Python provides useful functions such as `sum()` and `len()`:

```
grades = [1.7, 2.3, 1.3, 2.0]

average = sum(grades) / len(grades)
print("Average grade:", round(average, 2))
```

```
Average grade: 1.82
```

5.6 Expanding Lists: `.append()`

Sometimes you don't know the list elements beforehand — you can **add new values later**:

```
names = []

names.append("Ali")
names.append("Bente")

print(names)
```

```
['Ali', 'Bente']
```

Note

The `.append()` method adds a new value to the end of the list.

5.7 Nested Loops

If you're working with **multidimensional data** — such as a table with several rows — you can **nest loops**:

```
weekdays = ["Mon", "Tue", "Wed"]
hours = [1, 2, 3]

for day in weekdays:
    for hour in hours:
        print(f"{day}, hour {hour}")
```

```
Mon, hour 1
Mon, hour 2
Mon, hour 3
Tue, hour 1
Tue, hour 2
Tue, hour 3
Wed, hour 1
Wed, hour 2
Wed, hour 3
```

This results in:

```
Mon, hour 1
Mon, hour 2
Mon, hour 3
Tue, hour 1
...
```

5.8 Sorting Lists

With `sorted()`, you can sort lists **alphabetically or numerically**:

```
names = ["Zoe", "Anna", "Ben"]
sorted_names = sorted(names)

print(sorted_names)
```

```
['Anna', 'Ben', 'Zoe']
```

Important

The original list remains **unchanged**.

If you want to modify the list directly, you can use:

```
names.sort()
```

6 Reusable Code with Functions

Imagine you need to perform a specific calculation multiple times in your program. Instead of writing the same code repeatedly, you can bundle it into a **function**.

Functions are a key tool to make code:

- clear,
- reusable, and
- testable.

6.1 Learning Goals of This Chapter

By the end of this chapter, you will be able to:

- create your own functions using `def`,
- pass parameters and use return values,
- apply functions effectively in programs.

6.2 Defining a Function

A function consists of the following parts:

1. **Definition** with `def`
2. **Function name**
3. **Parameters in parentheses (optional)**
4. **Indentation** for the function body
5. (optional) **return statement**

Example:

```
def hello(name="Guest"):
    greeting = "Hello " + name + "!"
    return greeting
```

Let's start with the first keyword. Functions are defined with **def** and can be called as often as you like:

```
def greeting():  
    print("Hello and welcome!")
```

It only runs when you call it:

```
greeting()
```

Hello and welcome!

6.3 Passing Parameters

Functions can receive input values (parameters):

```
def greeting(name):  
    print("Hello", name + "!")  
  
greeting("Alex")
```

Hello Alex!

6.4 Return Values with return

A function can also **return** a value:

```
def square(number):  
    return number * number  
  
result = square(5)  
print(result)
```

6.5 Example: Conversions

6.5.1 Euro to US Dollar

```
def euro_to_usd(amount_euro):  
    exchange_rate = 1.09  
    return amount_euro * exchange_rate  
  
print("20 € is", euro_to_usd(20), "US dollars.")
```

20 € is 21.8 US dollars.

Task: Greeting with Name

Create a function `greet(name)` that uses the name in a greeting text:

```
Hello Fatima, nice to see you!
```

Solution

```
def greet(name):  
    print("Hello", name + ", nice to see you!")  
  
greet("Fatima")
```

Hello Fatima, nice to see you!

Task: Temperature Conversion

Write a function that converts Celsius to Fahrenheit:

Formula: $[F = C \times 1.8 + 32]$

Solution

```
def celsius_to_fahrenheit(c):  
    return c * 1.8 + 32  
  
print(celsius_to_fahrenheit(20))
```

68.0

6.6 Parameters with Default Values

You can assign **default values** to parameters. That way, the function can be called even without providing a value:

```
def greeting(name="Guest"):
    print("Hello", name + "!")

greeting()          # Hello Guest!
greeting("Maria")   # Hello Maria!
```

```
Hello Guest!
Hello Maria!
```

print() vs. return

These two concepts are often confused:

Expression	Meaning
<code>print()</code>	displays text on the screen
<code>return</code>	gives a value back to the caller

Example:

```
def double(x):
    return x * 2

# Make the output visible
print(double(5)) # Output: 10
```

```
10
```

7 Working with Files

Programs often work not only with user input but also with **text files** – for example, to save or load data.

Python offers simple functions to:

- **open** files,
- **read from** or **write to** them,
- and **close** them afterward.

7.1 Learning Goals of This Chapter

By the end of this chapter, you will be able to:

- open files using `open()`,
- read content from text files,
- write text into files,
- work safely and easily using `with` blocks.

7.2 Reading a File

```
# Example: Reading a file
with open("01-daten/beispiel.txt", "r") as file:
    content = file.read()
    print(content)
```

This is a sample file.
It contains several lines of text.
Each line can be read individually.

- `"r"` stands for **read**.
- `with` ensures that the file is automatically closed after reading.
- `read()` reads the **entire content** of the file as a string.

7.3 Reading Line by Line

```
with open("01-daten/beispiel.txt", "r") as file:
    for line in file:
        print("Line:", line.strip())
```

Line: This is a sample file.
Line: It contains several lines of text.
Line: Each line can be read individually.

Note

`.strip()` removes whitespace and line breaks at the beginning and end.

Task: Read a File

Suppose there is a file `gruesse.txt` with the following content:

Hallo Anna
Guten Morgen Ben
Willkommen Carla

Write a program that reads each line one by one and outputs it using `print(...)`.

Solution

```
with open("01-daten/gruesse.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Hallo Anna
Guten Morgen Ben
Willkommen Carla

7.4 Writing to a File

```
with open("ausgabe.txt", "w") as file:
    file.write("This is a new line.\n")
    file.write("And another one.")
```

- "w" stands for **w**rite.
- Caution: An existing file will be **overwritten**!

7.5 Writing Line by Line Using a Loop

```
data = ["Apple", "Banana", "Cherry"]

with open("obst.txt", "w") as f:
    for entry in data:
        f.write(entry + "\n")
```

! Important

Each line ends with `\n` for a line break.

💡 Task: Write a List to a File

Given a list of cities:

```
cities = ["Berlin", "Hamburg", "Munich"]
```

- Write a program that writes each city to a new line in a file called `cities.txt`.

Solution

```
cities = ["Berlin", "Hamburg", "Munich"]

with open("cities.txt", "w") as f:
    for city in cities:
        f.write(city + "\n")
```

7.6 Reading All Lines at Once with `readlines()`

Instead of iterating over a file, you can read all lines at once as a list:

```
with open("01-daten/beispiel.txt", "r") as f:
    lines = f.readlines()
    print(lines)
```

```
['This is a sample file.\n', 'It contains several lines of text.\n', 'Each line can be read :']
```

! Important

Each line ends with `\n`, so post-processing with `.strip()` is often helpful:

```
for line in lines:
    print(line.strip())
```

```
This is a sample file.
It contains several lines of text.
Each line can be read individually.
```

7.7 Closing Files Manually with `close()`

If you **do not** use a **with block**, you must close the file yourself – otherwise it stays open:

```
file = open("test.txt", "w")
file.write("This is a test.")
file.close()
```

! Important

`close()` is important to ensure changes are saved and the file is not locked.

Recommendation: Always use `with open(...)`, because Python will automatically close the file – even if an error occurs.